PHASE 15 OF 18

JVM Deep Dive & Modern Java (8 → 25)

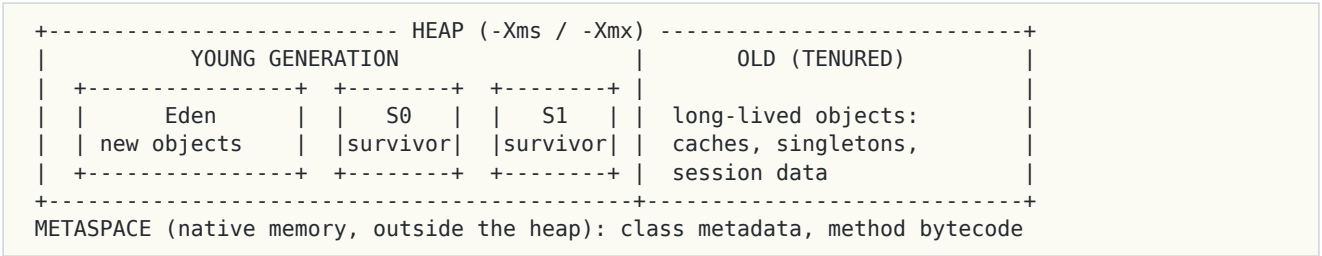
Heap generations and garbage collection, GC roots, collectors and tuning, reference types, class loading in depth, lambdas, functional interfaces, method references, Streams and Collectors, Optional, java.time, records, sealed classes, a version feature map and a rapid-fire interview round

IN THIS PHASE

15.1 Heap Structure and Object Lifecycle
15.2 How Garbage Collection Works
15.3 Class Loading in Depth
15.4 Lambda Expressions (Java 8)
15.5 Built-in Functional Interfaces (java.util.function)
15.6 Method References
15.7 Streams API
15.8 Optional (Java 8)
15.9 Date and Time API
15.10 Records (Java 16)
15.11 Sealed Classes (Java 17) and Pattern Matching
15.12 Modern Java Feature Map (8 → 25)
15.13 Rapid-Fire Interview Round (all phases)
15.14 Quick Revision

15.1 Heap Structure and Object Lifecycle

Most objects **die young** (temporary strings, DTOs, iterators). The HotSpot heap is therefore split into **generations**, so the GC can clean short-lived objects cheaply and often.



1. New objects are allocated in **Eden**.
2. When Eden fills up, a **Minor GC** runs. Live objects are copied to a Survivor space (S0 or S1), and Eden is wiped in one go.
3. Survivors bounce between S0 and S1. Each GC they survive increases their **age**.
4. Objects that reach the tenuring threshold (age up to 15) are **promoted** to the **Old generation**. Very large objects may go there directly.
5. When the Old generation fills, a **Major/Full GC** runs. It is slower, with longer pauses.

GC event	Cleans	Typical cost
Minor / Young GC	Young generation	Frequent, fast (copying live objects only)
Major / Old GC	Old generation	Less frequent, slower
Full GC	Whole heap (+ Metaspace)	Slowest; long pauses: a red flag in production

15.2 How Garbage Collection Works

GC roots and reachability

An object is **alive** if it is **reachable** from a **GC root** through a chain of strong references. Everything else is garbage, including cycles (the **island of isolation**).

GC roots include: local variables and parameters in active thread stacks, static fields of loaded classes, active threads, JNI references, and objects used as locks by running code.

Mark, Sweep, Compact

```
Before GC:      [A] [B] [C] [D] [E]          A, C, E reachable; B, D unreachable
1. Mark:        [A*][B ][C*][D ][E*]         walk from GC roots, mark live objects
2. Sweep:       [A ][  ][C ][  ][E ]         reclaim the memory of unmarked objects
3. Compact:     [A ][C ][E ][  free  ]       slide survivors together, no fragmentation
```

- **Marking** traverses the object graph from the GC roots.
- **Sweeping** reclaims unmarked memory, which can leave the heap **fragmented**.
- **Compaction** moves live objects together so large allocations still fit, updating references.
- **Copying** (used in the young generation) copies live objects to a fresh space and implicitly compacts.

Not every collector does these steps the same way; modern collectors do most of the marking **concurrently**, while your application keeps running.

Stop-the-World pauses

During some GC phases, the JVM pauses **all application threads** (a **Stop-the-World**, or STW, pause) so object references stay consistent. Long STW pauses cause latency spikes and timeouts, for example a payment API that suddenly takes 2 seconds. Choosing and tuning the collector is mostly about **pause time vs throughput**.

Garbage collectors in HotSpot

Collector	Flag	Style	Best for
Serial	`-XX:+UseSerialGC`	Single-threaded, STW	Small heaps, containers with 1 CPU, client tools
Parallel (Throughput)	`-XX:+UseParallelGC`	Multi-threaded, STW	Batch jobs where total throughput matters more than pauses
G1 (Garbage-First)	`-XX:+UseG1GC` (**default since Java 9**)	Region-based, mostly concurrent marking, pause-time target	General-purpose servers (`-XX:MaxGCPauseMillis=200`)
ZGC	`-XX:+UseZGC` (generational in 21+)	Concurrent, sub-millisecond pauses, huge heaps	Low-latency services (trading, payments)
Shenandoah	`-XX:+UseShenandoahGC`	Concurrent compaction, low pause	Low-latency (in OpenJDK builds)
Epsilon	`-XX:+UseEpsilonGC`	No-op: never collects	Performance testing, very short-lived jobs

> **NOTE**
> The CMS collector (Concurrent Mark Sweep) was deprecated in Java 9 and **removed in Java 14**. If you see `-XX:+UseConcMarkSweepGC` in old scripts, it must be migrated.

Diagnosing memory problems

```
-Xms2g -Xmx2g                          # equal min/max avoids resize pauses (common in prod)
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/var/dumps
-Xlog:gc*:file=gc.log:time,uptime      # unified GC logging (Java 9+)

jcmd <pid> GC.heap_info                # heap usage
jcmd <pid> GC.heap_dump /tmp/h.hprof   # heap dump -> analyse in Eclipse MAT / VisualVM
jstat -gcutil <pid> 1000               # GC stats every second
jcmd <pid> JFR.start duration=60s filename=rec.jfr   # Java Flight Recorder profiling
```

Common memory leaks in Java: ever-growing `static` collections or caches with no eviction, listeners that are never removed, `ThreadLocal` values left on pool threads, unclosed resources, inner-class instances holding outer objects, and mutable `HashMap` keys.

Reference types

Reference	Collected when	Use
Strong (`Obj o = new Obj()`)	Never, while reachable	Normal code
`SoftReference`	Only when the JVM is low on memory	Memory-sensitive caches
`WeakReference`	At the next GC once weakly reachable	`WeakHashMap`, canonicalising maps, listener registries
`PhantomReference`	After finalization; `get()` always returns `null`	Post-mortem cleanup (`Cleaner`)

15.3 Class Loading in Depth

The class loader hierarchy

Loader	Loads	`getClassLoader()` **returns**
Bootstrap (native code)	Core modules: `java.base` (`java.lang`, `java.util`, ...)	`null`
Platform (named *Extension* before Java 9)	Other JDK modules (`java.sql`, `java.xml`)	`PlatformClassLoader`
Application / System (`AppClassLoader`)	Your classpath / module path: app classes and JARs	`AppClassLoader`
Custom loaders	Plugins, app servers, hot reloading, isolation	Your loader

```
System.out.println(String.class.getClassLoader());          // null (bootstrap)
System.out.println(java.sql.Date.class.getClassLoader());   // ...PlatformClassLoader
System.out.println(MyApp.class.getClassLoader());           // ...AppClassLoader
System.out.println(ClassLoader.getSystemClassLoader().getParent());   // PlatformClassLoader
```

The parent delegation model

```
loadClass("com.bank.Payment")
   Application loader: already loaded? no -> ask parent
      Platform loader: already loaded? no -> ask parent
         Bootstrap loader: in java.base? no -> return "not found"
      Platform loader: in its modules? no
   Application loader: search the classpath -> FOUND -> define the class
   (if no loader finds it -> ClassNotFoundException)
```

Why delegate upwards?

- **Security**: nobody can replace `java.lang.String` with a malicious copy on the classpath.
- **Uniqueness**: each class is loaded once by the right loader.
- **Consistency**: a class's identity is *its name plus its defining loader*, so the same class loaded by two loaders gives two distinct types (classic `ClassCastException: X cannot be cast to X` in app servers).

Loading → Linking → Initialization

Phase	What happens
Loading	Find the `.class` bytes, create the `Class` object in Metaspace
Linking: **Verification**	Bytecode verifier checks format and type safety (`VerifyError`)
Linking: **Preparation**	Allocate static fields and set them to **default values** (0, null, false)
Linking: **Resolution**	Symbolic references → direct references (may be lazy)
Initialization	Run static initialisers and `static { }` blocks in textual order (once, thread-safe)

A class is **initialised** on first *active use*: `new`, calling a static method, reading or writing a non-constant static field, `Class.forName(name)`, initialising a subclass, or being the main class. Merely declaring a variable of that type, using `Xyz.class`, or reading a compile-time constant (`static final int X = 5`) does **not** trigger initialisation.

```
Class<?> c1 = Class.forName("com.mysql.cj.jdbc.Driver");  // loads AND initialises
Class<?> c2 = ClassLoader.getSystemClassLoader()
                         .loadClass("com.bank.Payment");  // loads only; init deferred
```

> **TIP**
> The JIT compiler is covered in Phase 1. HotSpot interprets first, profiles, and compiles hot methods with C1 (fast) and then C2 (optimised). Microbenchmarks without a warm-up phase (JMH) are therefore misleading.

15.4 Lambda Expressions (Java 8)

A **lambda** is an anonymous function: a concise implementation of a **functional interface** (one abstract method). It enables passing **behaviour as data**.

```
// (parameters) -> expression      or      (parameters) -> { statements; }
Runnable r       = () -> System.out.println("run");
Comparator<String> byLen = (a, b) -> Integer.compare(a.length(), b.length());
Predicate<Integer> isEven = n -> n % 2 == 0;           // one parameter: parentheses optional
Function<Integer, Integer> square = (Integer x) -> x * x;   // explicit type (optional)
BinaryOperator<Integer> add = (var a, var b) -> a + b;      // var in lambdas (Java 11)
Supplier<List<String>> listMaker = () -> {               // block body needs return
    List<String> l = new ArrayList<>();
    l.add("init");
    return l;
};
```

- The **target type** (the functional interface the lambda is assigned to) determines the parameter and return types.
- Captured local variables must be **effectively final**.
- Inside a lambda, `this` refers to the **enclosing** instance (unlike in anonymous classes).
- Lambdas don't create a `.class` file; they are linked at runtime via `invokedynamic`.

15.5 Built-in Functional Interfaces (java.util.function)

Interface	Method	Takes → Returns	Example
`Predicate<T>`	`test(T)`	T → boolean	`s -> s.isBlank()`
`Function<T,R>`	`apply(T)`	T → R	`s -> s.length()`
`Consumer<T>`	`accept(T)`	T → void	`s -> System.out.println(s)`
`Supplier<T>`	`get()`	() → T	`() -> LocalDate.now()`
`UnaryOperator<T>`	`apply(T)`	T → T	`s -> s.toUpperCase()`
`BinaryOperator<T>`	`apply(T, T)`	(T, T) → T	`(a, b) -> a + b`
`BiFunction<T,U,R>`	`apply(T, U)`	(T, U) → R	`(acct, amt) -> acct + ":" + amt`
`BiPredicate<T,U>`, `BiConsumer<T,U>`	`test` / `accept`	Two inputs	`(k, v) -> ...`
Primitive versions	`IntPredicate`, `IntFunction`, `ToIntFunction`, `IntUnaryOperator`, `IntSupplier`, ...	Avoid boxing	`IntPredicate p = i -> i > 0;`

```
Predicate<String> notBlank = s -> !s.isBlank();
Predicate<String> isNumeric = s -> s.chars().allMatch(Character::isDigit);
Predicate<String> validAccount = notBlank.and(isNumeric).and(s -> s.length() == 10);
validAccount.test("1234567890");          // true
notBlank.negate().test("  ");             // true
Predicate.not(String::isBlank);           // Java 11

Function<Double, Double> addGst   = amt -> amt * 1.18;
Function<Double, Double> discount = amt -> amt - 100;
addGst.andThen(discount).apply(1000.0);   // (1000 * 1.18) - 100 = 1080.0
addGst.compose(discount).apply(1000.0);   // (1000 - 100) * 1.18 = 1062.0
Function.identity();                      // x -> x
```

15.6 Method References

A **method reference** (`Class::method`) is shorthand for a lambda that only calls an existing method.

Kind	Syntax	Equivalent lambda
Static method	`Integer::parseInt`	`s -> Integer.parseInt(s)`
Instance method of a particular object	`System.out::println`	`x -> System.out.println(x)`
Instance method of an arbitrary object of a type	`String::toUpperCase`	`s -> s.toUpperCase()`
Constructor	`ArrayList::new`, `int[]::new`	`() -> new ArrayList<>()`, `n -> new int[n]`

```
List<String> ids = List.of("3", "1", "2");
ids.stream().map(Integer::parseInt).sorted().forEach(System.out::println);   // 1 2 3
Supplier<Map<String, Integer>> mapMaker = HashMap::new;
Function<String, StringBuilder> sbMaker = StringBuilder::new;
```

15.7 Streams API

A **Stream** is a sequence of elements supporting **declarative**, **pipelined** operations (filter, map, reduce...). You describe *what* you want; the library handles the *how*.

- A stream is **not a data structure**: it doesn't store elements, it processes them from a source.
- It **doesn't modify** the source.
- It is **lazy**: intermediate operations run only when a terminal operation is invoked.
- It is **single-use**: after a terminal operation, reusing it throws `IllegalStateException`.

```
SOURCE           INTERMEDIATE OPERATIONS (lazy, return Stream)       TERMINAL (triggers work)
list.stream() -> filter(...) -> map(...) -> sorted(...) -> limit(n) -> collect / forEach / sum
```

Creating streams

```
List.of(1, 2, 3).stream();                    // from a collection
Arrays.stream(new int[]{1, 2, 3});            // IntStream from an array
Stream.of("a", "b", "c");                     // from values
IntStream.range(1, 5);                        // 1 2 3 4   (rangeClosed(1, 5) includes 5)
Stream.iterate(1, n -> n * 2).limit(5);       // 1 2 4 8 16
Stream.iterate(1, n -> n <= 100, n -> n * 3); // 1 3 9 27 81 (Java 9)
Stream.generate(UUID::randomUUID).limit(3);   // 3 random ids
Files.lines(path);  "a,b".chars();  map.entrySet().stream();
```

Intermediate operations (lazy)

Operation	Purpose
`filter(Predicate)`	Keep matching elements
`map(Function)`	Transform each element
`mapToInt/Long/Double`	To a primitive stream (`sum`, `average`, ...)
`flatMap(Function<T, Stream<R>>)`	Flatten nested structures (a list of lists → one stream)
`distinct()`	Remove duplicates (uses `equals`)
`sorted()` / `sorted(Comparator)`	Sort
`limit(n)` / `skip(n)`	Truncate / skip
`peek(Consumer)`	Look at elements (debugging)
`takeWhile` / `dropWhile` (9)	Take or drop while a condition holds (on ordered streams)
`boxed()`	`IntStream` → `Stream<Integer>`

Terminal operations

Operation	Returns
`forEach(Consumer)`	void
`collect(Collector)` / `toList()` (16)	A collection or other result
`reduce(identity, BinaryOperator)`	A single combined value
`count()`, `min(cmp)`, `max(cmp)`	long / `Optional<T>`
`sum()`, `average()`, `summaryStatistics()`	Primitive streams only
`anyMatch`, `allMatch`, `noneMatch`	boolean (short-circuit)
`findFirst()`, `findAny()`	`Optional<T>` (short-circuit)
`toArray()`	Array

Streams in practice

```
record Txn(String id, String account, String type, double amount, String city) { }

List<Txn> txns = List.of(
    new Txn("T1", "SB-1", "CREDIT", 5000, "Pune"),
    new Txn("T2", "SB-1", "DEBIT",  1200, "Pune"),
    new Txn("T3", "SB-2", "CREDIT", 9000, "Delhi"),
    new Txn("T4", "SB-3", "DEBIT",  300,  "Delhi"),
    new Txn("T5", "SB-2", "DEBIT",  7000, "Mumbai"));

// High-value transaction ids, largest first
List<String> big = txns.stream()
        .filter(t -> t.amount() >= 5000)
        .sorted(Comparator.comparingDouble(Txn::amount).reversed())
        .map(Txn::id)
        .toList();                                          // [T3, T5, T1]

double totalCredit = txns.stream()
        .filter(t -> t.type().equals("CREDIT"))
        .mapToDouble(Txn::amount)
        .sum();                                             // 14000.0

boolean anyHuge = txns.stream().anyMatch(t -> t.amount() > 8000);      // true
Optional<Txn> smallest = txns.stream().min(Comparator.comparingDouble(Txn::amount));  // T4
List<String> cities = txns.stream().map(Txn::city).distinct().sorted().toList();
// [Delhi, Mumbai, Pune]

int sum = IntStream.rangeClosed(1, 100).sum();                         // 5050
int product = Stream.of(1, 2, 3, 4).reduce(1, (a, b) -> a * b);         // 24
DoubleSummaryStatistics st = txns.stream().mapToDouble(Txn::amount).summaryStatistics();
// count=5, sum=22500.0, min=300.0, average=4500.0, max=9000.0

List<List<String>> nested = List.of(List.of("a", "b"), List.of("c"));
List<String> flat = nested.stream().flatMap(List::stream).toList();     // [a, b, c]
```

Collectors

```
import static java.util.stream.Collectors.*;

Map<String, List<Txn>> byCity = txns.stream().collect(groupingBy(Txn::city));
Map<String, Long> countByType = txns.stream().collect(groupingBy(Txn::type, counting()));
// {CREDIT=2, DEBIT=3}
Map<String, Double> sumByAccount = txns.stream()
        .collect(groupingBy(Txn::account, TreeMap::new, summingDouble(Txn::amount)));
// {SB-1=6200.0, SB-2=16000.0, SB-3=300.0}
Map<Boolean, List<String>> partition = txns.stream()
        .collect(partitioningBy(t -> t.amount() > 2000, mapping(Txn::id, toList())));
// {false=[T2, T4], true=[T1, T3, T5]}
String idsCsv = txns.stream().map(Txn::id).collect(joining(", ", "[", "]"));
// [T1, T2, T3, T4, T5]
Map<String, Double> amountById = txns.stream().collect(toMap(Txn::id, Txn::amount));
Map<String, Double> maxByCity = txns.stream()
        .collect(toMap(Txn::city, Txn::amount, Math::max));     // merge fn for duplicate keys
Map<String, Double> avgByType = txns.stream()
        .collect(groupingBy(Txn::type, averagingDouble(Txn::amount)));
// {CREDIT=7000.0, DEBIT=2833.33...}
```

Collector	Result
`toList()`, `toSet()`, `toCollection(TreeSet::new)`	Collections
`toMap(keyFn, valueFn[, mergeFn[, mapFactory]])`	Map (**duplicate keys without a merge function →** `IllegalStateException`)
`groupingBy(classifier[, mapFactory][, downstream])`	`Map<K, List<T>>` or a downstream result

Collector	Result
`partitioningBy(predicate[, downstream])`	`Map<Boolean, ...>`
`counting()`, `summingInt/Double`, `averagingX`, `minBy`, `maxBy`	Aggregates (often downstream)
`mapping(fn, downstream)`, `filtering`, `flatMapping` (9)	Transform inside a group
`joining(delim[, prefix, suffix])`	String
`collectingAndThen(c, finisher)`	Post-process (for example make it unmodifiable)
`teeing(c1, c2, merger)` (12)	Combine two collectors

Laziness and short-circuiting (verified)

```
Stream.of("a", "bb", "ccc", "dddd")
      .filter(s -> { System.out.println("filter " + s); return s.length() > 1; })
      .map(s ->    { System.out.println("map " + s);    return s.toUpperCase(); })
      .findFirst();
// filter a
// filter bb
// map bb        <- stops here: elements flow one at a time; ccc and dddd are never touched
```

Stream vs Collection, and pitfalls

Collection	Stream
Stores data	Computes over data
Eager	Lazy
Can be traversed many times	Single-use
External iteration (you write the loop)	Internal iteration (the library loops)
Mutable (for most)	Never modifies its source

PITFALL

Common stream mistakes:

- Reusing a stream → `IllegalStateException: stream has already been operated upon or closed`.
- Side effects in `map`/`filter` (modifying external lists). Use `collect` instead.
- `toMap` with duplicate keys and no merge function → `IllegalStateException`.
- `stream().toList()` is unmodifiable; `collect(Collectors.toList())` currently returns a mutable `ArrayList` (not guaranteed).
- `parallelStream()` for small or I/O-bound workloads, or with shared mutable state, is usually slower or wrong. Measure first.
- Forgetting that nothing executes without a terminal operation.

15.8 Optional (Java 8)

`Optional<T>` is a container that **may or may not hold a value**. It makes "might be absent" explicit in an API **return type**, instead of returning `null`.

```
Optional<String> some  = Optional.of("SB-1001");          // value must be non-null
Optional<String> maybe = Optional.ofNullable(lookup());   // null -> empty
Optional<String> none  = Optional.empty();

maybe.isPresent();  maybe.isEmpty();                       // checks (isEmpty: Java 11)
maybe.ifPresent(acc -> System.out.println("Found " + acc));
maybe.ifPresentOrElse(System.out::println, () -> System.out.println("none"));   // Java 9

String a = maybe.orElse("DEFAULT");                        // default (always evaluated)
String b = maybe.orElseGet(() -> expensiveDefault());      // lazy default
String c = maybe.orElseThrow(() -> new AccountNotFoundException("missing"));
String d = some.orElseThrow();                             // Java 10: NoSuchElementException

Optional<Integer> len = maybe.map(String::length);         // transform if present
Optional<String> pune = findCustomer(id).map(Customer::getAddress)   // safe navigation
        .map(Address::getCity)
        .filter(city -> city.equals("Pune"));
```

REMEMBER

Optional best practices:

- Use it mainly as a **method return type**, not for fields, parameters or collections (return an empty list instead).
- Never call `get()` without checking; prefer `orElse`/`orElseGet`/`orElseThrow`/`map`.
- `orElse(x)` always evaluates `x`; use `orElseGet` when the default is expensive.
- Never return `null` from a method declared to return `Optional`.

15.9 Date and Time API

Legacy classes: why they were replaced

- `java.util.Date` is **mutable**, its name is misleading (it is really an instant with time), and many methods are deprecated. Its `toString` uses the JVM's time zone.
- `java.sql.Date` extends `java.util.Date` and represents only a date, for JDBC.
- `Calendar` (`Calendar.getInstance()`) is mutable and clunky, and **months are 0-based** (`Calendar.MONTH` = 0 is January): a famous bug source.
- `SimpleDateFormat` is **not thread-safe**: sharing one instance between threads gives corrupted dates.

```
Calendar c = Calendar.getInstance();
int month = c.get(Calendar.MONTH) + 1;          // +1 needed: 0-based months!
int hour = c.get(Calendar.HOUR_OF_DAY);
Date now = new Date();                          // legacy: avoid in new code

StringTokenizer st = new StringTokenizer("Java,Spring,Boot", ",");   // legacy tokenizer
while (st.hasMoreTokens()) System.out.println(st.nextToken());      // prefer String.split()
```

java.time (Java 8): immutable, thread-safe, clear

Class	Represents	Example
`LocalDate`	Date only, no time zone	`2026-08-15` (birth date, EMI due date)
`LocalTime`	Time only	`10:30:00` (branch opening time)
`LocalDateTime`	Date + time, no zone	`2026-08-15T10:30`
`ZonedDateTime`	Date + time + zone	`2026-08-15T10:30+05:30[Asia/Kolkata]`
`Instant`	A point on the UTC timeline	`2026-08-15T05:00:00Z` (timestamps, logs, DB)
`Duration`	Time-based amount (seconds, nanos)	`PT2H30M`
`Period`	Date-based amount (years, months, days)	`P1Y2M3D`
`DateTimeFormatter`	Formatting and parsing (thread-safe)	`dd-MM-yyyy`
`ChronoUnit`	Units for arithmetic and differences	`ChronoUnit.DAYS.between(a, b)`

```
LocalDate today = LocalDate.of(2026, 8, 15);                // months are 1-based (August = 8)
LocalDate due = today.plusMonths(1).withDayOfMonth(5);       // 2026-09-05
LocalDate monthEnd = today.with(TemporalAdjusters.lastDayOfMonth());   // 2026-08-31
today.getDayOfWeek();                                        // SATURDAY
today.isLeapYear();                                          // false
today.isBefore(due);                                         // true
// today.plusDays(1) returns a NEW object; 'today' itself never changes (immutable)

LocalDate dob = LocalDate.of(1990, 3, 25);
Period age = Period.between(dob, today);                     // P36Y4M21D
age.getYears();                                              // 36
long days = ChronoUnit.DAYS.between(today, due);             // 21

LocalTime open = LocalTime.of(9, 30);
Duration shift = Duration.between(open, LocalTime.of(17, 45));   // PT8H15M
LocalDateTime ldt = LocalDateTime.of(today, open);           // 2026-08-15T09:30

DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd-MM-yyyy");
String text = today.format(fmt);                             // 15-08-2026
LocalDate parsed = LocalDate.parse("05-09-2026", fmt);       // DateTimeParseException if invalid
LocalDate iso = LocalDate.parse("2026-08-15");               // ISO format by default

ZonedDateTime ist = ZonedDateTime.of(ldt, ZoneId.of("Asia/Kolkata"));
ZonedDateTime ny  = ist.withZoneSameInstant(ZoneId.of("America/New_York"));   // 00:00 same day
Instant stamp = ist.toInstant();                             // 2026-08-15T04:00:00Z

Date legacy = Date.from(stamp);                              // bridge legacy <-> modern
Instant back = legacy.toInstant();
```

Pattern letter	Meaning	Pattern letter	Meaning
`yyyy` / `uuuu`	Year	`HH` / `hh`	Hour 0–23 / 1–12
`MM` / `MMM` / `MMMM`	Month 08 / Aug / August	`mm`	**Minutes** (not months!)
`dd`	Day of month	`ss` / `SSS`	Seconds / milliseconds
`EEE` / `EEEE`	Mon / Monday	`a`	AM/PM

TIP

Store timestamps as `Instant` (UTC) in databases and logs; convert to `ZonedDateTime` only for display. Use `LocalDate` for calendar concepts (DOB, due dates). In tests, inject a `java.time.Clock` (`LocalDate.now(clock)`) so "today" is controllable and tests are deterministic.

15.10 Records (Java 16)

A **record** is a concise, **immutable data carrier**. The compiler generates private final fields, a canonical constructor, accessor methods (`name()`, not `getName()`), and `equals`, `hashCode` and `toString`.

```
public record Money(BigDecimal amount, String currency) {
    public Money {                                          // compact constructor: validation
        Objects.requireNonNull(amount, "amount");
        if (amount.signum() < 0) throw new IllegalArgumentException("negative amount");
        currency = currency.toUpperCase();                  // normalise a parameter
    }
    public static Money inr(String amt) { return new Money(new BigDecimal(amt), "INR"); }
    public Money plus(Money other) {                        // methods allowed
        return new Money(amount.add(other.amount), currency);
    }
}

Money m = Money.inr("250.00");
System.out.println(m);                                      // Money[amount=250.00, currency=INR]
System.out.println(m.amount());                             // 250.00
System.out.println(m.equals(Money.inr("250.00")));          // true (value-based equality)

record Customer(String id, List<String> phones) {
    Customer { phones = List.copyOf(phones); }              // defensive copy: truly immutable
}
```

Can	Cannot
Implement interfaces	Extend another class (records implicitly extend `java.lang.Record`)
Have static fields and methods, instance methods, nested types	Declare instance fields beyond the components
Customise the constructor (compact or canonical)	Have setters (fields are `final`)
Be generic: `record Pair<A,B>(A first, B second)`	Be extended (records are implicitly `final`)

> **NOTE**
> Records are ideal for DTOs, API request/response bodies (Jackson supports them), map keys, value objects (Money, AccountId) and multiple return values. JPA **entities** shouldn't be records, because entities need mutability and a no-arg constructor.

15.11 Sealed Classes (Java 17) and Pattern Matching

A **sealed** class or interface restricts **which classes may extend or implement it**, using `permits`. Each permitted subclass must be `final`, `sealed` or `non-sealed`. The compiler then knows **every possible subtype**, so `switch` can be checked for exhaustiveness.

```
public sealed interface PaymentResult permits Success, Failure, Pending { }
public record Success(String txnId, BigDecimal amount) implements PaymentResult { }
public record Failure(String code, String reason)      implements PaymentResult { }
public record Pending(String txnId)                    implements PaymentResult { }

static String describe(PaymentResult r) {
    return switch (r) {                                    // no default needed: exhaustive
        case Success s when s.amount().compareTo(new BigDecimal("100000")) > 0
                -> "Large payment " + s.txnId() + " flagged for review";
        case Success(String id, BigDecimal amt)            // record pattern (Java 21)
                -> "Paid " + amt + " (" + id + ")";
        case Failure(var code, var reason)
                -> "Failed [" + code + "]: " + reason;
        case Pending p -> "Awaiting confirmation for " + p.txnId();
    };
}
```

Adding a new permitted type (for example `Reversed`) makes every non-exhaustive `switch` **fail to compile**, so the compiler finds every place that must handle the new case.

15.12 Modern Java Feature Map (8 → 25)

Version	Year	Key language and API features
8 (LTS)	2014	Lambdas, functional interfaces, method references, Streams, `Optional`, `java.time`, default/static interface methods, `CompletableFuture`
9	2017	Modules (JPMS), JShell, `List.of`/`Set.of`/`Map.of`, private interface methods, `Optional`/Stream improvements, G1 default
10	2018	`var` for local variables, `List.copyOf`, `orElseThrow()`
11 (LTS)	2018	`String.isBlank`/`strip`/`lines`/`repeat`, `Files.readString`/`writeString`, standard `HttpClient`, `var` in lambdas, single-file launch
12–13	2019	`Collectors.teeing`, switch-expression and text-block previews
14	2020	**Switch expressions** (final), helpful NullPointerException messages
15	2020	**Text blocks** (final), ZGC production-ready
16	2021	**Records**, **pattern matching for** `instanceof`, `Stream.toList()`
17 (LTS)	2021	**Sealed classes**, strict floating point everywhere, new random generators
18	2022	**UTF-8 by default**, simple web server (`jwebserver`), `finalize` deprecated for removal
19–20	2022–23	Virtual threads and structured concurrency (preview), `HashMap.newHashMap(n)`
21 (LTS)	2023	**Virtual threads**, **pattern matching for switch**, **record patterns**, **sequenced collections**, generational ZGC
22	2024	**Unnamed variables and patterns** (_), FFM API (native interop), launching multi-file source programs
23–24	2024–25	Markdown doc comments, **Stream gatherers** (24), less virtual-thread pinning with `synchronized` (24)
25 (LTS)	2025	**Compact source files & instance** `main`, **flexible constructor bodies**, **scoped values**, **module import declarations**

```
// Java 22+: unnamed variable for things you must declare but don't use
try { process(); } catch (IOException _) { log("ignored"); }
map.forEach((_, value) -> total += value);
```

15.13 Rapid-Fire Interview Round (all phases)

Question	Short answer
JDK vs JRE vs JVM?	Dev kit ⊃ runtime ⊃ virtual machine
Is Java pass-by-reference?	No: always pass-by-value (references are copied)
`==` vs `equals()`?	Reference vs logical content
Why is String immutable?	Pool safety, security, thread safety, hash caching
String vs StringBuilder vs StringBuffer?	Immutable / mutable and fast / mutable and synchronized
Can we override a static method?	No: it is hidden
Can we overload `main`?	Yes; the JVM calls only `main(String[])`
Abstract class vs interface?	State and constructors, single inheritance / contract, multiple inheritance
Why no multiple inheritance of classes?	Diamond problem
`final` vs `finally` vs `finalize`?	Modifier / cleanup block / deprecated GC hook
Checked vs unchecked exceptions?	Compiler-enforced / RuntimeException and Error
`throw` vs `throws`?	Raise one / declare possible
Does finally always run?	Except `System.exit`, a crash, or a hang
`equals`/`hashCode` contract?	Equal objects ⇒ equal hash codes
HashMap internal working?	Hash → bucket → equals; tree bins at 8; resize at 0.75
HashMap vs ConcurrentHashMap?	Unsafe, 1 null key / thread-safe, no nulls, atomic ops
ArrayList vs LinkedList?	Array, O(1) get / nodes, O(n) get
Fail-fast vs fail-safe?	Throws CME / snapshot or weakly consistent
Comparable vs Comparator?	Natural order inside the class / external, many orders
`sleep()` vs `wait()`?	Keeps lock / releases monitor
`synchronized` vs `volatile`?	Mutual exclusion + visibility / visibility only
How do you prevent deadlock?	Consistent lock ordering, `tryLock` with timeout
Runnable vs Callable?	No result / result + checked exceptions
What is a daemon thread?	Background thread that doesn't keep the JVM alive
Platform vs virtual thread?	OS thread / cheap JVM thread for blocking I/O
What is `transient`?	Field skipped by serialization
Why `serialVersionUID`?	Version check during deserialization
Stack vs heap?	Per-thread frames / shared objects managed by GC
When is an object GC-eligible?	When unreachable from GC roots
Default GC since Java 9?	G1
Parent delegation?	Ask the parent loader first: security and uniqueness
`Class.forName` vs `loadClass`?	Loads and initialises / loads only
Intermediate vs terminal stream ops?	Lazy, return a Stream / trigger execution
`map` vs `flatMap`?	One-to-one / one-to-many, flattened
`orElse` vs `orElseGet`?	Eager default / lazy default
Record vs class?	Immutable data carrier with generated members
Why sealed classes?	Restricted hierarchy plus exhaustive switch
`LocalDateTime` vs `ZonedDateTime` vs `Instant`?	No zone / with zone / UTC point
Why BigDecimal for money?	Exact decimal arithmetic with rounding control

Question	Short answer
Integer cache range?	-128 to 127, so never compare wrappers with ==

15.14 Quick Revision

- Heap: **Eden → Survivors → Old**; Metaspace is outside the heap. Minor GC is frequent and cheap; Full GC is a warning sign.
- GC = reachability from **GC roots**; mark → sweep → compact; STW pauses; **G1** is the default, **ZGC** for low latency; CMS is removed.
- Tools: `-Xlog:gc`, heap dumps on OOM, `jcmd`, `jstat`, JFR, Eclipse MAT. Leaks: static caches, listeners, `ThreadLocal`, mutable keys.
- References: strong > soft (low memory) > weak (next GC) > phantom.
- Class loaders: Bootstrap (`null`) → Platform → Application; parent delegation. Load → Verify/Prepare/Resolve → Initialise (static blocks) on first active use.
- Lambdas implement functional interfaces; captured locals are effectively final; `this` = enclosing instance.
- `java.util.function`: Predicate, Function, Consumer, Supplier, Unary/BinaryOperator, Bi-versions, primitive versions; compose with `and`/`or`/`negate`/`andThen`/`compose`.
- Method references: static, bound instance, unbound instance, constructor.
- Streams: source → lazy intermediate ops → one terminal op; single-use; no source mutation. Collectors: `groupingBy`, `counting`, `summingX`, `partitioningBy`, `toMap` (with a merge function), `joining`.
- Optional: return types only; `map`/`filter`/`orElseGet`/`orElseThrow`; never a bare `get()`.
- java.time: immutable, thread-safe; `LocalDate`/`LocalTime`/`LocalDateTime`/`ZonedDateTime`/`Instant`; `Period` (dates) vs `Duration` (time); `DateTimeFormatter` (`MM` months, `mm` minutes).
- Legacy `Date`/`Calendar` (0-based months) and `SimpleDateFormat` (not thread-safe): avoid in new code.
- Records: immutable carriers, compact constructors, accessors without the `get` prefix. Sealed + records + pattern switch = exhaustive, type-safe domain modelling.
- LTS versions: **8, 11, 17, 21, 25**.